

This work is licensed under a Creative Commons license



Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Software Reverse Engineering

Part IV – Static & Dynamic Analysis

Simone Aonzo, PhD



Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - **YARA**
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- **Ghidra and PyGhidra Automation**

2 Dynamic Analysis

- In-guest
 - **x64dbg**
 - **Process Monitor**
- Out-of-guest
- Sandbox
 - **CAPE Sandbox**

Static Analysis

Inspect binaries *without* execution

- *Typical tasks*
 - Investigate how the file is structured (file format and metadata)
 - Build an abstraction to represent the program
 - Assign/recover meaningful names of functions and variables
- *Pros*
 - Fast and scalable — can analyze many binaries efficiently
 - No execution risk — safe for untrusted or malicious code
 - Provides global code visibility — all paths are theoretically accessible
- *Cons*
 - Path explosion (too many code paths to explore)
 - Fails to capture dynamically generated/executed code
 - It is deliberately made more difficult by **obfuscation**

Obfuscation

The deliberate process of making a program harder to understand or analyze – both for humans and automated tools – while preserving its semantics (i.e., without changing its behavior)

Dynamic Analysis

Observe a program's behavior *during execution* to understand its runtime actions and effects

- *Pros*

- Reveals ground-truth behavior
- Provides concrete values and real data flows
- Real side effects (network traffic, file writes, etc.)

- *Cons*

- Coverage gaps — only observes executed code paths
- Limited scalability (time-consuming)
- Difficult to reproduce non-deterministic behaviors
- The analysis environment can be detected and **evaded**

Evasive software

Designed to detect, avoid, or mislead analysis. Often it consists of exhibiting different behaviors in different environments [BCK⁺10]

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Step zero: equality and similarity

How can we check the “identity” of a file? **Hash functions**

$$h : \{0, 1\}^m \mapsto \{0, 1\}^n$$

Take an arbitrary number of m bits as input, produce a short fixed-size output of n bits, called hash value or **digest**

- **Cryptographic** (e.g., SHA-256) $\implies$ checking for equality (yes/no)
 - Avalanche effect
 - Collision resistance
 - ...
- **Fuzzy** (e.g., SSDEEP and TLSH) $\implies$ checking for similarity (score)
 - Output similar digests when the inputs are similar
 - Need a $cmp()$ function to compare digests and return a similarity score ss
 - $cmp(h_1, h_2) \mapsto ss \in \mathbb{N}$
 - The higher ss is, the more similar h_1 and h_2 are

Finding equal or similar files among many

Equality: fast to compare, secure, but easy to break (i.e., **hashbusting** – next slide)

- MD5 and SHA-1 are broken, but faster to compute than SHA-256
- SHA-256 is the safest choice for compliance and standardization
- BLAKE3 is the fastest secure crypto hash
- xxHash is an insanely fast non-crypto hash

Similarity: computationally expensive. With n files: $\binom{n}{2} = \frac{n^2-n}{2}$

- SSDEEP
 - Requires at least 4KB
 - Returns a similarity score $ss \in [0, 100]$
- TLSH
 - Requires at least 50B
 - Returns a distance $d \in [0, \infty)$
 - Normalized as similarity score $ss = \max(\frac{300-d}{3}, 0) \in [0, 100]$
 - Choose TLSH: more accurate and robust [VAD⁺25]

Cryptographic vs. Fuzzy hash – Example

```
from hashlib import sha256; import tlsh # pip3 install py-tlsh

h1 = sha256('Hello world!'.encode('utf-8')).hexdigest()
h2 = sha256('Hello world?'.encode('utf-8')).hexdigest()
assert h1 == 'c0535e4be2b79ffd93291305436bf889314e4a3faec05ecffcbb7df31ad9e51a'
assert h2 == '43f497ee7ac09843d631362ef9aca26a0cab437acaea8a98e44afa7ad65a2d41'

data = bytearray([i for i in range(50)]) # min 50 bytes for tlsh
h1 = tlsh.hash(bytes(data))
data[42] = 88 # one byte difference
h2 = tlsh.hash(bytes(data))
assert h1 == 'T1509004D4C7D44CCF5D1735CCD155045F554375F750C41030073105D54F55554C71151C'
assert h2 == 'T10E9004D445D44CCFDD1715CCD0550C7D554375F750C01030173105D54F41554C75113C'
assert tlsh.diff(h1, h2) == 24
```

Hashbusting

Making a (usually small) edit to a file so that its cryptographic hash changes, while maintaining the same functionality

Outline

1 Static Analysis

- Equality and Similarity
- **Structural Characteristics**
- Signature-Based Detection
 - **YARA**
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- **Ghidra and PyGhidra Automation**

2 Dynamic Analysis

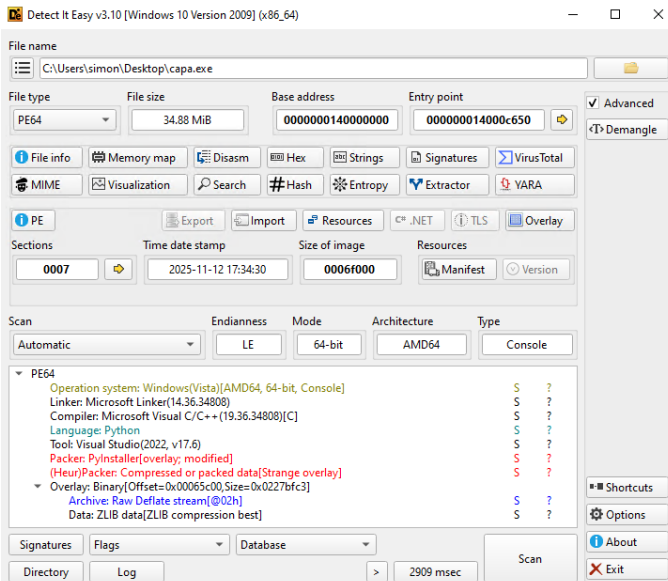
- In-guest
 - **x64dbg**
 - **Process Monitor**
- Out-of-guest
- Sandbox
 - **CAPE Sandbox**

Step one: file type and structure

See “*How to determine the file type?*” in “*OS Mechanisms*”, plus:
Jack of all trades: **Detect It Easy**

- Links
 - Engine: <https://github.com/horsicq/DIE-engine/>
 - Signatures: <https://github.com/horsicq/Detect-It-Easy/>
 - Fine-grained and open-source
- Key features
 - Signatures for file types, linker, compiler, protectors/packers
 - Memory Map, i.e., Sections
 - Hashing
 - Entropy
 - Strings

Detect It Easy



Entropy: A Measure of Uncertainty

The average amount of uncertainty or surprise you expect when observing the outcome of a discrete random variable X , with possible outcomes $x_1, \dots, x_n$, which occur with probability $P(x_1), \dots, P(x_n)$, the Shannon entropy of X is formally defined as:

$$H(x) = - \sum_{i=1}^n P(x_i) \log_2(P(x_i))$$

With 1 Byte = 8 bits, then $2^8 = 256$ outcomes

$$n = 256 \implies 0 \leq H(x) \leq 8$$

It measures

- How much information you gain when the result is revealed
- How surprised you should be by the outcome
- The average minimum number of bits needed to encode the outcome

The Divine Entropy (1/2)

```
> head commedia.txt
```

```
LA DIVINA COMMEDIA  
di Dante Alighieri  
INFERNO
```

Inferno: Canto I

Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscura
ché la diritta via era smarrita.

```
> zip commedia.txt.zip commedia.txt  
adding: commedia.txt (deflated 60%)
```

```
> ls -al  
total 772  
drwxrwxr-x  2 simo simo  4096 ago 31 10:28 .  
drwxr-xr-x 120 simo simo  4096 ago 31 10:28 ..  
-rw-rw-r--  1 simo simo 557960 ago 31 10:21 commedia.txt  
-rw-rw-r--  1 simo simo 220610 ago 31 10:27 commedia.txt.zip
```

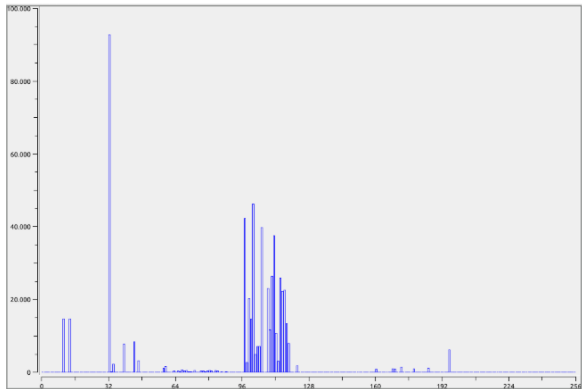
```
> xxd commedia.txt | head
```

```
00000000: 4c41 2044 4956 494e 4120 434f 4d4d 4544  LA DIVINA COMMED  
00000010: 4941 0d0a 6469 2044 616e 7465 2041 6c69  IA..di Dante Ali  
00000020: 6768 6965 7269 0d0a 494e 4645 524e 4f0d  ghieri..INFERNO.  
00000030: 0a0d 0a0d 0a49 6e66 6572 6e6f 3a20 4361  ....Inferno: Ca  
00000040: 6e74 6f20 490d 0a0d 0a20 204e 656c 206d  nto I.... Nel m  
00000050: 657a 7a6f 2064 656c 2063 616d 6d69 6e20  ezzo del cammin  
00000060: 6469 206e 6f73 7472 6120 7669 7461 0d0a  di nostra vita..  
00000070: 6d69 2072 6974 726f 7661 6920 7065 7220  mi ritrovai per  
00000080: 756e 6120 7365 6c76 6120 6f73 6375 7261  una selva oscura  
00000090: 0d0a 6368 c3a9 206c 6120 6469 7269 7474  ..ch.. la diritt
```

```
> xxd commedia.txt.zip | head
```

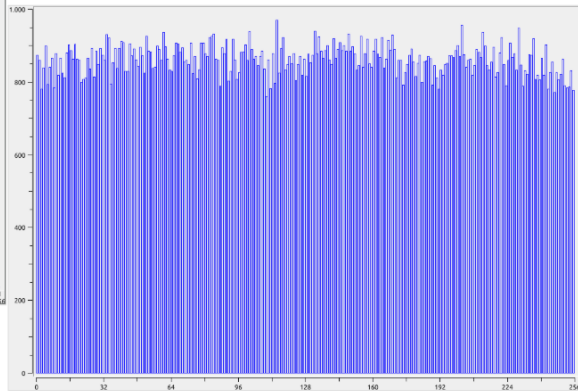
```
00000000: 504b 0304 1400 0000 0800 ad52 1f55 fd26  PK.....R.U.&  
00000010: 8f54 145d 0300 8883 0800 0c00 1c00 636f  .T.].....co  
00000020: 6d6d 6564 6961 2e74 7874 5554 0900 0306  mmedia.txtUT....  
00000030: 1a0f 6307 1a0f 6375 780b 0001 04e8 0300  ..C...cux.....  
00000040: 0004 e803 0000 acfd b9b2 1cd7 b92d 8cfa  ....-...  
00000050: 8ad0 3b24 9753 4e6d 3d00 6030 4080 e465  ..;$.$Nm=.`0@.e  
00000060: 043b 09d8 8c13 c7cb 5535 abd6 8cc8 a694  .;.....US.....  
00000070: 4d19 30ee b3c8 d48e 90c5 6d1d 1b2f 76e7  M.0.....m./v.  
00000080: 68be 9959 2024 ebea ffcf 26b0 506b 5636  h..Y $....&.PkV6  
00000090: b3f9 9ad1 fcf8 a679 f7c3 6f3f fcfc a679  ....y..o?..y
```

The Divine Entropy (2/2)



commedia.txt
 $H = 4.486$

commedia.txt.zip
 $H = 7.998$



The Divine Encryption

Recipe

AES Encrypt

Key

63 34 23 ac 21 ...

HEX ▾

IV

06 7f ac 57 21 ...

HEX ▾

Mode

CBC

Input

Raw

Output

Raw

Entropy

Visualisation

Shannon scale

Input

LA DIVINA COMMEDIA%
di Dante Alighieri%
INFERNO%

Inferno: Canto I%

Nel mezzo del cammin di nostra vita%
mi ritrovai per una selva oscura%
ché la diritta via era smarrita.%
Ah! quanto a dir qual era è cosa dura%
esta selva selvaggia e aspra e forte%
che nel pensier rinova la paura!%
Tant'è amara che poco è più morte;%
ma per trattar del ben ch'i' vi trovai,%
dirò de l'altre cose ch'i' v'ho scorte.%
Io non so ben ridir com'i' v'intrai,%
tant'era pien di sonno a quel punto%

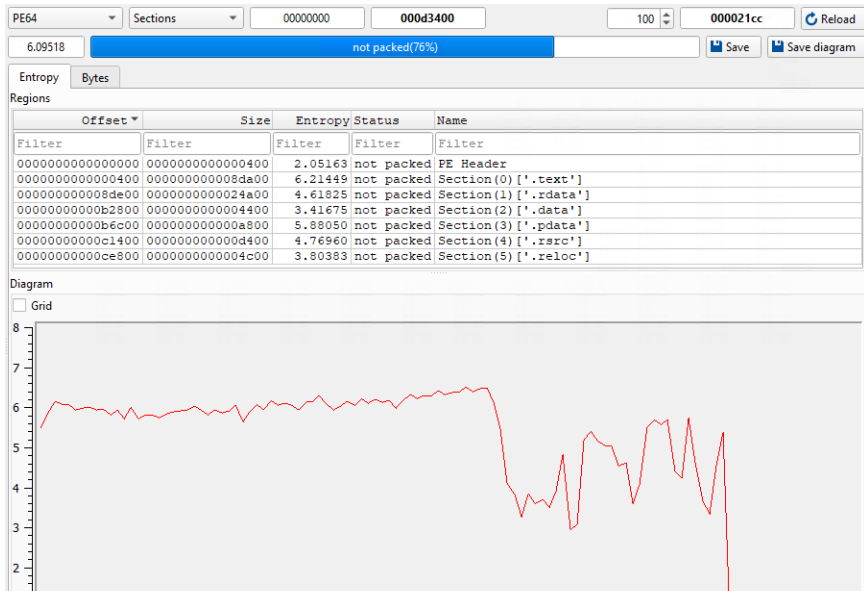
551846

14754

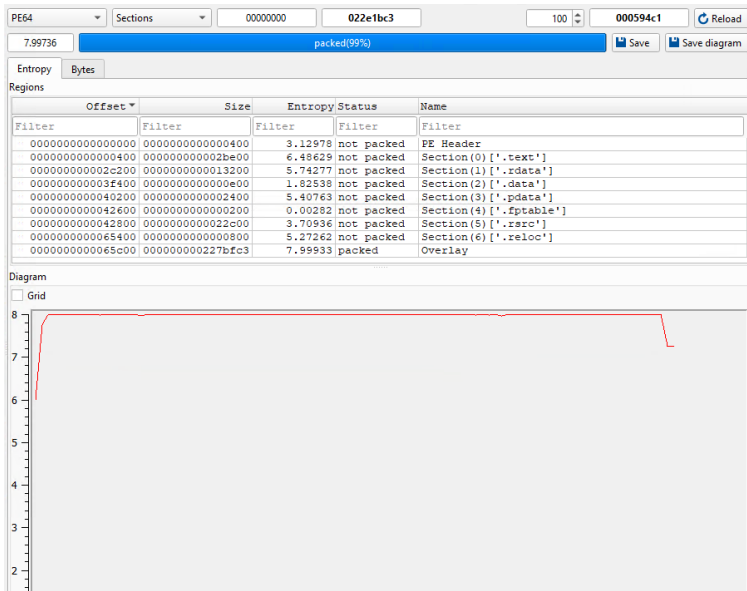
Output

Shannon entropy: 7.999724053703733

Detect It Easy – Entropy – Example (1/2)



Detect It Easy – Entropy – Example (2/2)



Ghidra will be explained later, but from now on...

Ghidra: This is a custom block with colors matching the Ghidra logo.
As a reference, these blocks will describe how to find/use some of Ghidra's features.

Strings

Sequences of readable characters (ASCII/Unicode) inside a binary file

- Provide fast insights, e.g.
 - Network: domains, URLs, IPs, ...
 - Persistence: registry keys, dropped files, ...
 - OS interaction: APIs, shell commands, ...
- Indicate obfuscation or packing when few/no strings are present
- Great place to start looking at the code
 - That is, code where “interesting” strings are referenced

Tools (in addition to Detect It Easy)

- Linux strings (one encoding mode at a time 😞)
- strings by MS Sysinternals ¹
- FLOSS by Mandiant (de-obfuscator) ²

Ghidra: Window → Defined Strings

¹<https://learn.microsoft.com/en-us/sysinternals/downloads/strings>

²<https://github.com/mandiant/flare-floss>

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Signature-Based Detection

Idea

Identify malicious software by matching **reusable indicators** extracted from known samples/families

- **What can be a signature?**
 - strings: mutexes, paths, domains, ...
 - byte patterns: characteristic code fragments
 - structural properties: imports, sections, resources, metadata
- **Typical workflow**
 - reverse engineer one or more samples
 - extract stable indicators
 - combine them into a rule
- **Rule:** a declarative specification of indicators and matching logic

Strengths

- fast to apply
- explainable detections
- precise for known families

Limitations

- brittle under obfuscation
- too specific $\Rightarrow$ misses variants
- too generic $\Rightarrow$ false positives

Goal: combine weak signals into one robust detection rule

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - **YARA**
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- **Ghidra and PyGhidra Automation**

2 Dynamic Analysis

- In-guest
 - **x64dbg**
 - **Process Monitor**
- Out-of-guest
- Sandbox
 - **CAPE Sandbox**

The pattern matching Swiss knife for malware researchers



It is in another slide deck... [[link](#)] ➡

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Disassembling

Process of parsing a binary stream (with valid machine code) and convert it back to assembly instructions

Challenges 🤖

- Variable-length instructions, like x86
 - While ARM has a fixed length (except ARM32 in Thumb mode)
- Code vs. data ambiguity
- Indirect jumps/calls

Instructions are organized into appropriate **data structures** to facilitate:

- Analysis for algorithms
- Understanding for humans

Ghidra: Window → Listing

Basic Block (BB)

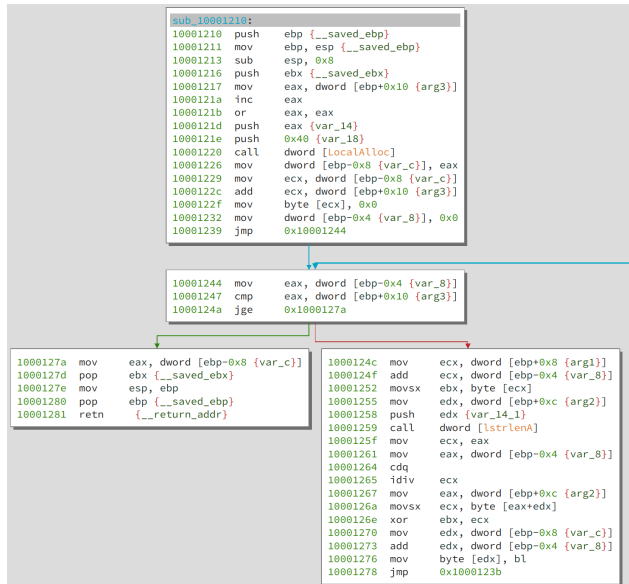
A maximal sequence of instructions with one entry and one exit, where control can only enter at the beginning and leave at the end

- No jumps or branches **into** the middle
- No jumps or branches **out of** the middle (only at the end)
- Execution enters at the beginning and leaves at the end without halting or branching

Does the x86 `call` instruction end a BB?

- In theory YES: it temporarily transfers control
- But most disassemblers use a slightly different convention
- *BB ends at instructions whose next instruction is not statically known*
 - A `jmp`, `ret`, or conditional branch $\rightarrow$ ends a BB
 - the next address is uncertain
 - A `call` does not end a BB
 - the next instruction (i.e., the address right after the call) is known

Basic Block (BB) – Example



Subroutine / Function

Subroutines and functions are reusable blocks of code designed to perform a specific task and return control to the caller

- In theory, Function $\implies$ returns a value
- In practice are often used interchangeably
 - E.g., `sub_` prefix of IDA Pro and BinaryNinja, while `FUN_` of Ghidra
- Composed of:
 - Entry address
 - List of basic blocks
 - Frame pointer
 - Parameters
 - Local variables

On x86, the frame pointer is usually stored in the (E|R)BP register

Parameters at positive offsets: e.g., [EBP + 8]

Local variables at negative offsets: e.g., [EBP - 4]

... some optimizations may use the (E|R)SP

Subroutine / Function – Example (32bit)

```
*****
*                                     *
*                                     FUNCTION                                     *
*                                     *
*****
undefined4 __cdecl FUN_0040684c(undefined4 param_1, undefined4 * param_2, undefined4 * param_3)
    assume FS_OFFSET = 0xffdf000
    undefined4 EAX:4    <RETURN>
    undefined4 Stack[0x4]:4 param_1
                                XREF[2]: 00406cac (R),
                                00406ce2 (R)
    undefined4 * Stack[0x8]:4 param_2
                                XREF[1]: 0040686d (R)
    undefined4 * Stack[0xc]:4 param_3
                                XREF[1]: 00406a84 (R)
    undefined4 Stack[-0xa54... local_a54
                                XREF[1]: 00406988 (R)
    undefined4 Stack[-0xa5c... local_a5c
                                XREF[1]: 00406990 (R)
    undefined4 Stack[-0xa60... local_a60
                                XREF[1]: 00406994 (R)
    undefined4 Stack[-0xa68... local_a68
                                XREF[1]: 00406998 (R)
    undefined4 Stack[-0xa6c... local_a6c
                                XREF[2]: 0040688b (R),
                                00406892 (R)
    undefined Stack[-0xa78... local_a78
                                XREF[1]: 00406882 (*)
    undefined Stack[-0xa9c... local_a9c
                                XREF[1]: 0040698c (*)
    undefined4 Stack[-0xab0... local_ab0
                                XREF[1]: 00406899 (W)
    undefined4 Stack[-0xab4... local_ab4
                                XREF[2]: 004068a1 (W),
                                004068ae (*)
    undefined4 Stack[-0xac4... local_ac4
                                XREF[1]: 00406875 (W)
    FUN_0040684c
                                XREF[1]: FUN_00406ebc:00407208(c)
0040684c 55          PUSH     EBP
0040684d 8b ec       MOV      EBP,ESP
0040684f 83 e4 f8     AND       ESP,0xffffffff8
```

- PUSH EBP; MOV EBP, ESP → prologue
- AND ESP, 0xffffffff8 → stack alignment to a multiple of 8 bytes

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Graphs are the best data structures to describe a program...

- Call Graph (CG)
- Control Flow Graph (CFG)
 - Inter Procedural CFG (IPCFCG)
- Data Flow Graph (DFG)
 - Inter Procedural DFG (IPDFG)

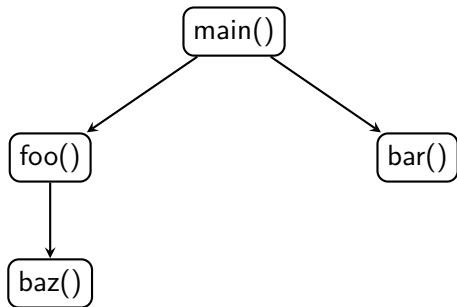
Inter Procedural analysis

⇒ the analysis spans the entire program
i.e., all its Subroutines|Functions, rather than a single one

Call Graph (CG)

Directed graph that represents calling relationships between Subroutines|Functions

- Scope: program
- Nodes represent Subroutines|Functions
- Edges indicate a call from one to another



Ghidra: Window → Function Call Graph

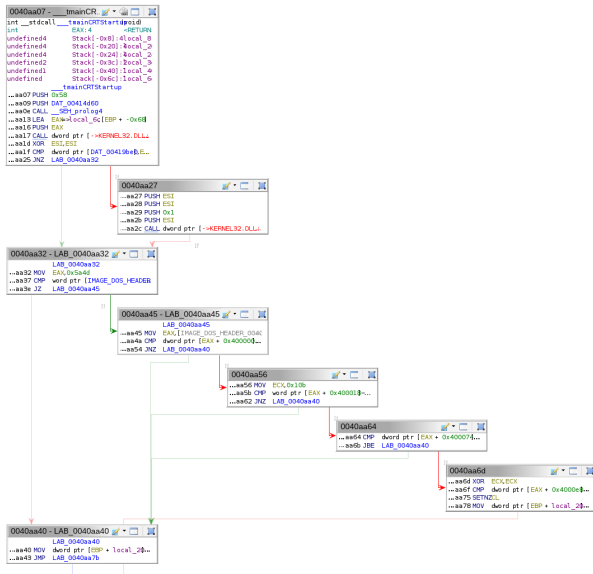
Control Flow Graph (CFG)

Directed graph that models all possible paths that execution can take within a single Subroutine|Function

- Scope: Subroutine|Function
- Nodes represent basic blocks (BBs)
- Edges represent control flow between them

Ghidra: Window → Function Graph

Control Flow Graph (CFG) – Example



Data Flow Graph (DFG)

Directed graph that represents data dependencies between operations or variables

- Scope: Subroutine|Function
- Nodes represent operations or variables
- Edges represent data dependencies

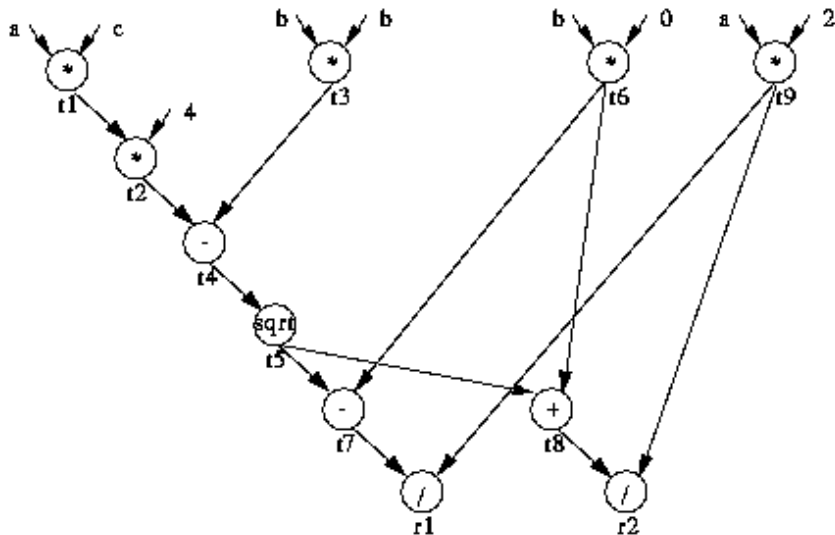
DFG is for machines 🤖

While DFGs can also help humans, they provide a structured representation that is very useful for algorithms for dependency analysis:

what influences a given variable?

Data Flow Graph (DFG) – Example

```
quad(a, b, c):  
  t1 = a * c  
  t2 = 4 * t1  
  t3 = b * b  
  t4 = t3 - t2  
  t5 = sqrt(t4)  
  t6 = -b  
  t7 = t6 - t5  
  t8 = t7 + t5  
  t9 = 2 * a  
  r1 = t7 / t9  
  r2 = t8 / t9  
  return (r1, r2)
```



Disassembling Algorithms

Goal: recover instructions $\rightarrow$ basic blocks $\rightarrow$ functions $\rightarrow$ IPCFG

- **Linear sweep**

- Decode sequentially from a start address
- **Pros:** fast, simple **Cons:** mis-sync on data, junk code

- **Recursive traversal** (a.k.a. recursive descent):

- Follow control-flow targets
- **Pros:** fewer false decodes **Cons:** may miss unreachable/indirect code

- **Hybrid / Speculative**

- Combine sweep + traversal, heuristics, and metadata

This is attack surface!

Since disassembly is another undecidable problem, there are *anti-disassembly* techniques for deliberately misleading disassemblers

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Lifting

Machine instructions $\rightarrow$ architecture-*independent* **Intermediate Representation** (IR)

- Preserve semantics
 - registers, flags, memory effects, and control/data flow
- Normalized form
 - same representation of operations, regardless of the original ISA

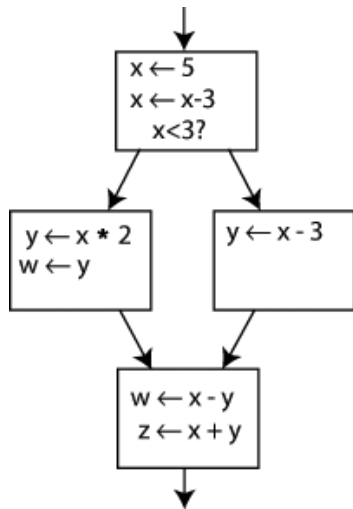
Why lifting?

- ISAs are designed primarily for efficient hardware execution
- IRs are designed for program analysis and transformation
 - $\implies$ better de-compilation (in a few slides...)

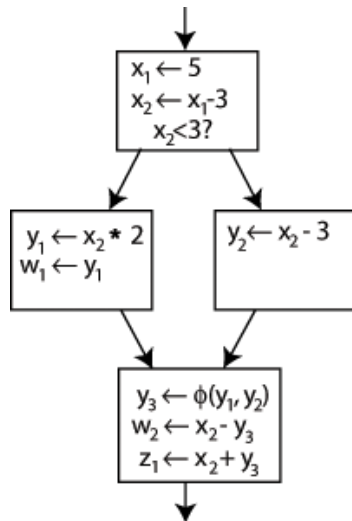
Types, i.e., two orthogonal design dimensions

- ① Structural representation $\rightarrow$ how operations are expressed
 - e.g. Three-Address Code (TAC)
 - `dst = src1 OP src2`
- ② Data-flow discipline $\rightarrow$ how values/variables are defined and used
 - e.g. Static Single Assignment (SSA)
 - Each variable is assigned *exactly once*
 - At control-flow joins, use ϕ to merge definitions

Three-Address Code and Static Single Assignment



TAC but not SSA



TAC and SSA

Ghidra's P-code (1/2)

During disassembly, for each instruction, Ghidra generates the raw P-code. It is essentially a typed, low-level TAC IR. E.g.:

```
10001239 : JMP 0x10001244
    ---  BRANCH (ram, 0x10001244, 4)

[...]

10001244 : MOV EAX,dword ptr [EBP + -0x4]
    (unique, 0x3300, 4) INT_ADD (register, 0x14, 4) , (const, 0xffffffffc, 4)
    (unique, 0xba00, 4) LOAD (const, 0x1a1, 4) , (unique, 0x3300, 4)
    (register, 0x0, 4) COPY (unique, 0xba00, 4)
```

Ghidra's P-code (2/2)

During the decompilation: TAC P-code → into SSA. Referred to as the **High Pcode**. E.g.:

```
10001232 c7 45 fc 00 00 00 00    MOV        dword ptr [EBP + local_8],0x0
10001239 eb 09                    JMP        LAB_10001244

LAB_1000123b
1000123b 8b 55 fc                    MOV        EDX,dword ptr [EBP + local_8]
1000123e 83 c2 01                    ADD        EDX,0x1
10001241 89 55 fc                    MOV        dword ptr [EBP + local_8],EDX

LAB_10001244
10001244 8b 45 fc                    MOV        EAX,dword ptr [EBP + local_8]
```

```
(ram, 0x10001232) : (stack, 0xf[...],8, 4) COPY (const, 0x0, 4)
(ram, 0x10001239) : --- BRANCH (ram, 0x10001244, 1)
(ram, 0x1000123e) :
    (register, 0x8, 4) INT_ADD (stack, 0xf[.]8, 4) , (const, 0x1, 4)
(ram, 0x10001244) :
    (stack, 0xf[.]8, 4) MULTIEQUAL (stack, 0xf[.]8, 4) , (register, 0x8, 4)
```

MULTIEQUAL is Ghidra's SSA representation of ϕ -function

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Decompiling

Goals

- **Semantic Fidelity:** Recover semantically equivalent source code
- **Syntactic Correctness:** The decompiled source code is re-compilable

Challenges:

- The compilation process is lossy
 - Compilation objective is performance not readability
- Lost symbols/types, optimizations, inlined/stripped code, etc.

Core steps:

- Disassembly → Lifting → Decompiling
- Type/ABI inference for variables/arguments recovery
- High-level structuring (loops, conditionals, etc.) and abstractions

Why? decompiled C code can be much shorter and simpler!

Ghidra: Window → Decompile: fun_x

Disassembling vs Decompiling – (extreme) Example

```
; ===== SUBROUTINE =====  
; Attributes: bp-based frame  
; mod_ll(long long)  
                                public _Z6mod_llx  
                                _Z6mod_llx proc near  
  
var_C      = dword ptr -0Ch  
arg_0      = qword ptr 8  
                                var_10      = dword ptr -10h  
  
    push    ebp  
    mov     ebp, esp  
    push    ebx  
    sub     esp, 0Ch  
    mov     ecx, dword ptr [ebp+arg_0]  
    mov     ebx, dword ptr [ebp+arg_0+4]  
    mov     eax, ecx  
    mov     edx, ebx  
    mov     eax, edx  
    sar     edx, 1Fh  
    sar     eax, 1Fh  
    mov     eax, edx  
    mov     edx, 0  
    shr     eax, 1Fh  
    add     eax, ecx  
    adc     ebx, ebx  
    shrd    eax, edx, 1  
    sar     edx, 1  
    mov     [ebp+var_10], eax  
    mov     [ebp+var_C], edx  
    mov     eax, [ebp+var_10]  
    mov     edx, [ebp+var_C]  
    shld    edx, eax, 1  
    add     eax, eax  
    sub     ecx, eax  
    sbb     ebx, edx  
    mov     [ebp+var_10], ecx  
    mov     [ebp+var_C], ebx  
    mov     eax, [ebp+var_10]  
    mov     edx, [ebp+var_C]  
    add     esp, 0Ch  
    pop     ebx  
    pop     ebp  
    retn  
_Z6mod_llx endp
```

```
__int64 __cdecl mod_ll(__int64 a1)  
{  
    return a1 % 2;  
}
```

Decompilation is still an active research field, look at [SYL25]

Famous decompilers

... or frameworks with built-in decompilation support

- **Hex-Rays** (of IDA Pro) 🏰
 - Lifts to IDA microcode (SSA-based IR)
- **Ghidra**
 - Lifts to Ghidra's P-code, then SSA transformation
- **BinaryNinja** — Multi-level IL stack
 - LLIL (Low-Level IL, faithful)
 - MLIL (Medium-Level IL, data-flow oriented)
 - HLIL (High-Level IL, C-like)
- **angr** [SWS⁺16]
 - Lifts to VEX IR used by the Valgrind (mature, about 20 years!)
 - Valgrind is dynamic binary instrumentation (DBI) used for memory debugging/profiling
 - Then, SSA transformation

You can compare their output: <https://dogbolt.org/>

Why is there no universal IR?

There is no single IR that is simultaneously:

- Architecture-faithful
- Analysis-friendly
- Optimization-friendly
- Decompilation-friendly
- Readable
- Fast to execute
- Easy to maintain

Faithfulness vs Abstraction

- **Low-level IR:** preserves exact machine behavior, but it is hard to structure
- **High-level IR:** easy to structure, but may lose subtle semantics

You cannot perfectly maximize both

Outline

1 Static Analysis

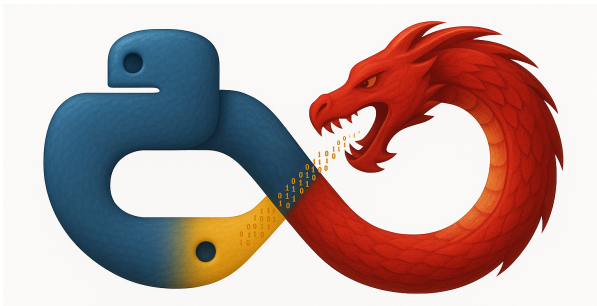
- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox



It is in another slide deck... [[link](#)] ➡



It is in another slide deck... 🖱️

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

Dynamic Analysis – Taxonomy

Runtime Environment

- Virtual Machine (VM) – virtualized or emulated hardware
- Bare Metal – a physical machine

Analysis Component

- In-guest
 - User-space
 - Debugger
 - Dynamic Binary Instrumentation (DBI)
 - Kernel-space
 - Module (Linux) / Driver (Windows)
- Out-of-guest
 - Hypervisor/Emulator Introspection

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

[In-guest / User-space] Debugger

It attaches to a running process (or launches one)

Features:

- Breakpoints (SW or HW) and watchpoints
- Memory and registers inspection
- Single-stepping through instructions or functions
- Code patching and runtime modification

Takeaways:

- Designed for manual human interaction (but it can be scripted)
- **Pros:** Fine-grained control over program execution
- **Cons:** Intrusive $\implies$ easily detected

Popular debuggers:

- Linux: GNU Debugger (GDB)
- Windows: WinDbg (from Microsoft), x64dbg
- Multiplatform: LLDB Debugger (LLDB)

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox



It is in another slide deck... ➡ [[link](#)]

[In-guest / User-space] Dynamic Binary Instrumentation

Insert analysis code into a *running* program **without recompiling** it

- DBI frameworks *rewrite* code at runtime to inject callbacks before/after selected instructions or APIs
- Just-In-Time code translation into an instrumented version
 - In a dedicated memory zone named Code Cache
- It executes the code in the cache instead of the original bytes

High-Level Architecture

- You write the **callback** (on some conditions) to log|modify behavior
- DBI inserts the **hook** that redirects execution
- Original binary → DBI engine → *instrumented* version executes
- When a condition occurs, the corresponding hook is executed

[In-guest / User-space] Dynamic Binary Instrumentation

Pros

- **Fine-grained:** control at instruction-level
- **Automated:** designed to be programmed

Cons

- **Overhead:** JIT + hooks slow execution (often 2–50×)
- **Stability:** complex apps can break
- **Intrusive** $\implies$ easily detected

Popular DBI frameworks:

- Intel Pin (Win/Linux, x86 only)
 - closed-source, dev in C++
- DynamoRIO (Win/Linux/macOS, x86 and ARM)
 - dev in C (C++ as well but with caveats)
- Frida (Win/Linux/macOS/Android/iOS, x86 and ARM)
 - limited instruction-level control; dev in JavaScript (Python bindings)

[In-guest / Kernel-space] Module/Driver (1/2)

A **kernel module/driver** runs inside the OS kernel

- When a user-space program interacts with the kernel → *event*
- From such modules/drivers you can register callbacks or hooks to intercept events, and eventually perform actions
- Used for Monitoring and Policy Enforcement
 - E.g., firewall and antivirus

Pros

- **Scope:** all user-space processes of the system
- **Efficacy:** difficult to detect and “impossible” to bypass from a user-space program

Cons

- **Granularity:** actions only in the request/response with the kernel
 - E.g., no instruction-level visibility
- **Stability:** bugs can crash the OS (BSOD/panic)
 - E.g., CrowdStrike Falcon outage in 2024 (8.5M Windows devices)

[In-guest / Kernel-space] Module/Driver (2/2)

Popular modules/drivers “frameworks”

- Windows:
 - Active: Minifilter (file, registry, I/O) and Windows Filtering Platform (WFP) Callout (network)
 - Passive: Event Tracing for Windows (ETW)
- Linux: kprobes/kretprobes, tracepoints, and ftrace
- Cross-platform: extended Berkeley Packet Filter (eBPF)
 - Programmable engine that orchestrates the other mechanisms

Representative implementations:

- Windows:
 - *Procmon* (Sysinternals) – kernel minifilter for event tracing
 - *Sysmon* – ETW-based service for telemetry
- Linux:
 - Procmon and Sysmon: eBPF-based, developed by Microsoft!
 - *Falco* – eBPF-based behavioral monitoring and threat detection

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - **YARA**
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- **Ghidra and PyGhidra Automation**

2 Dynamic Analysis

- In-guest
 - **x64dbg**
 - **Process Monitor**
- Out-of-guest
- Sandbox
 - **CAPE Sandbox**

Process Monitor



It is in another slide deck... 🖱️ [[link](#)]

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

[Out-of-guest] Hypervisor/Emulator Introspection

Observe a guest OS *from outside* the hypervisor/emulator

- **Virtual Machine Introspection (VMI)**
 - E.g., LibVMI for Xen, used by DRAKVUF
- **Emulator hooks/APIs:** instrument execution in emulator
 - E.g., QEMU TCG Plugins, used by PANDA

Pros:

- No guest modifications $\implies$ High stealth
- Powerful time-travel (record/replay) for root-cause analysis

Cons:

- *Semantic gap*: raw data $\rightarrow$ OS objects (costly to reconstruct)
- Performance overhead
- Platform coupling (hypervisor-specific APIs, maintenance)

Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox

A full dynamic analysis system that orchestrates execution, monitoring, and reporting of untrusted programs securely

- Runtime Environment
 - Usually Virtual Machine-based
 - Can also be bare metal for evasive malware
 - Reset to a clean state between runs
- Analysis Component
 - Combines multiple in-guest monitors (user and kernel)
 - Sometimes out-of-guest introspection
- Isolation: separated from other machines
- Network: air-gapped, fake services (e.g. flare-fakenet-ng), online

Sandbox Characteristics

- **Reset** – how easy it is to revert the sandbox to a pristine state
- **Containment** – how easy it is to contain the malicious activity performed by the sample
- **Instrumentation** – how easy it is to collect fine-grained information about the running sample
- **Overhead** – how much the sample execution is slowed down by the sandbox/instrumentation
- **Scalability** – how easy it is to run multiple samples in parallel
- **Fidelity** – how well the system resembles a real target

Sandbox Comparison

Criterion	Bare Metal	Virtual Machine	Emulator
Reset	✗	✓	✓
Instrumentation	✗	✓	★
Overhead	✓	✓	✗
Scalability	✗	★	✓
Fidelity	★	✗	✗

Rule of Thumb:

- Bare metal to avoid evasion
- Emulator for very detailed information (or multi-architecture)
- VMs for everything else

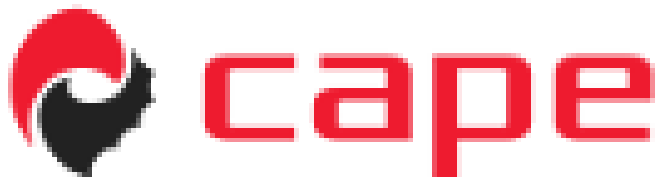
Outline

1 Static Analysis

- Equality and Similarity
- Structural Characteristics
- Signature-Based Detection
 - YARA
- Disassembling
- Call/Control/Data Graphs
- Lifting
- Decompiling
- Ghidra and PyGhidra Automation

2 Dynamic Analysis

- In-guest
 - x64dbg
 - Process Monitor
- Out-of-guest
- Sandbox
 - CAPE Sandbox



It is in another slide deck... ➡ [[link](#)]

- [BCK⁺10] Davide Balzarotti, Marco Cova, Christoph Karlberger, Engin Kirda, Christopher Kruegel, and Giovanni Vigna.
Efficient detection of split personalities in malware.
In *NDSS*, 2010.
- [SWS⁺16] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna.
SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis.
In *IEEE Symposium on Security and Privacy*, 2016.
- [SYL25] Melih Sirlanci, Carter Yagemann, and Zhiqiang Lin.
An empirical study of c decompilers: Performance metrics and error taxonomy.
In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 1707–1723, 2025.

- [VAD⁺25] Antonino Vitale, Simone Aonzo, Savino Dambra, Nanda Rani, Lorenzo Ippolito, Platon Kotzias, Juan Caballero, and Davide Balzarotti.
The polymorphism maze: Understanding diversities and similarities in malware families.
In *European Symposium on Research in Computer Security*, pages 505–525.
Springer, 2025.